

CENTRO DE ENSEÑANZA TÉCNICA Y SUPERIOR



Escuela de Ingeniería

Machine Learning

Unit 1 - Laboratory 2

Teacher: Cesar Alan Torres Quiroz

Presented by:

Martin Alejandro Hermosillo Vargas - T028361

Tijuana, Baja California, September 3rd, 2025

PART A: MALWARE SCANNER POSTERIOR

Use Bayes theorem to find the probability that a flagged file is actually malware.

```
[5] p_malware = 0.005
    p_flagged_given_malware = 0.95
    p_flagged_given_clean = 0.03

    Python

[6] p_flagged = (p_flagged_given_malware * p_malware) + (p_flagged_given_clean * (1 - p_malware))
    p_malware_given_flagged = (p_flagged_given_malware * p_malware) / p_flagged

    print(f'P(flagged) = {p_flagged:.5f}')
    print(f'P(malware | flagged) = {p_malware_given_flagged:.4f}')

    Python

... P(flagged) = 0.03460
    P(malware | flagged) = 0.1373
```

```
[7] n_files = 1_000_000
    true_flags = n_files * p_malware * p_flagged_given_malware
    false_flags = n_files * (1 - p_malware) * p_flagged_given_clean

    print(f'True flags: {true_flags:,.0f}')
    print(f'False flags: {false_flags:,.0f}')
    print(f'Ratio: {true_flags / (true_flags + false_flags):.4f}')

    Python

... True flags: 4,750
    False flags: 29,850
    Ratio: 0.1373
```

```
[8] np.random.seed(42)
    n_sim = 250_000
    is_malware = np.random.rand(n_sim) < p_malware

    flag_probs = np.where(is_malware, p_flagged_given_malware, p_flagged_given_clean)
    flagged = np.random.rand(n_sim) < flag_probs

    simulated_posterior = is_malware[flagged].mean()
    print(f'Simulated P(malware | flagged) = {simulated_posterior:.4f}')

    Python

... Simulated P(malware | flagged) = 0.1386
```

Part A short answer: A 95% detection rate measures sensitivity— $P(\text{flagged} \mid \text{malware})$ —meaning the scanner successfully catches 95% of actual threats. However, because clean files dominate the server (99.5%), even a low false-positive rate of 3% produces roughly 29,850 false alarms per 1,000,000 files compared to only 4,750 true detections. Thus, the actual likelihood that a flagged file is malicious is only about 13.74%. Conflating sensitivity with the posterior probability is an example of the base-rate fallacy.

PART B: MLE AND MAP IN A PHISHING DRILL

Use the beta bernoulli update from session 4. The prior is beta (2, 8), and the drill records 6 clicks and 18 ignores.

```
[9] alpha0, beta0 = 2, 8
    clicks, ignores = 6, 18

Python

n = clicks + ignores
mle = clicks / n
posterior_alpha = alpha0 + clicks
posterior_beta = beta0 + ignores
posterior_mean = posterior_alpha / (posterior_alpha + posterior_beta)
map_estimate = (posterior_alpha - 1) / (posterior_alpha + posterior_beta - 2)

print(f'MLE           = {mle:.3f}')
print(f'Posterior      = Beta({posterior_alpha}, {posterior_beta})')
print(f'Posterior mean  = {posterior_mean:.3f}')
print(f'MAP            = {map_estimate:.3f}')

[10] ... MLE           = 0.250
    Posterior      = Beta(8, 26)
    Posterior mean  = 0.235
    MAP            = 0.219
```

```
zero_clicks, zero_ignores = 0, 12

zero_mle = zero_clicks / (zero_clicks + zero_ignores)
zero_posterior_mean = (alpha0 + zero_clicks) / (alpha0 + zero_clicks + beta0 + zero_ignores)

print(f'Zero-click MLE           = {zero_mle:.3f}')
print(f'Zero-click posterior mean = {zero_posterior_mean:.3f}')

[11] ... Zero-click MLE           = 0.000
    Zero-click posterior mean = 0.091
```

Part B short answer: The posterior estimate is pulled toward the prior mean because Bayesian updating combines empirical observation with prior knowledge, treating the prior as initial pseudo-data. The Beta(2, 8) prior represents 10 virtual observations ($\alpha_0 + \beta_0 = 2 + 8$) with an expected click rate of 20%. In small sample sizes, such as the 0/12 drill, this prior acts as a regularizer, preventing extreme claims like a 0% click rate by blending the 12 observed trials with the 10 virtual trials to yield a realistic posterior mean of 9.09%.

PART C: CHOOSING POLYNOMIAL DEGREE WITH CROSS-VALIDATION.

Degree 1 is a straight line. The Pipeline below uses LinearRegression to estimate the coefficients for each polynomial degree. You do not need to derive the fitting procedure.

```
np.random.seed(42)
x = np.sort(np.random.uniform(0, 1, 40))
y = np.sin(2 * np.pi * x) + 0.25 * np.random.normal(size=40)
X = x.reshape(-1, 1)
x_grid = np.linspace(0, 1, 300).reshape(-1, 1)
```

[12]

Python

```
degrees = np.arange(1, 13)
train_mse, cv_means, cv_se = [], [], []
cv = KFold(n_splits=5, shuffle=True, random_state=42)

for degree in degrees:
    model = Pipeline([
        ('poly', PolynomialFeatures(degree=degree)),
        ('linear', LinearRegression()),
    ])

    model.fit(X, y)
    y_pred = model.predict(X)
    train_mse.append(mean_squared_error(y, y_pred))

    fold_scores = -cross_val_score(model, X, y, scoring='neg_mean_squared_error', cv=cv)
    cv_means.append(np.mean(fold_scores))
    cv_se.append(np.std(fold_scores, ddof=1) / np.sqrt(len(fold_scores)))
```

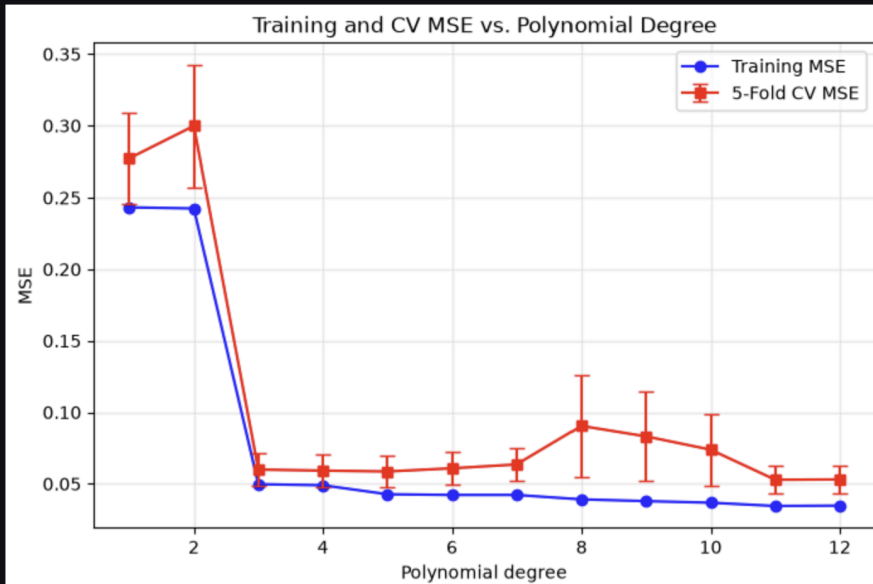
[13]

Python

```
plt.figure(figsize=(8, 5))
plt.plot(degrees, train_mse, label='Training MSE', color='blue', marker='o')
plt.errorbar(degrees, cv_means, yerr=cv_se, label='5-Fold CV MSE', color='red', marker='s', capsize=4)
plt.xlabel('Polynomial degree')
plt.ylabel('MSE')
plt.title('Training and CV MSE vs. Polynomial Degree')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

[14]

Python



```
selected_index = np.argmin(cv_means)
selected_degree = degrees[selected_index]

print(f'Selected degree: {selected_degree}')
print(f'CV MSE: {cv_means[selected_index]:.4f} +/- {cv_se[selected_index]:.4f}')
```

[15]

Python

```
... Selected degree: 11
CV MSE: 0.0528 +/- 0.0097
```

```

plot_degrees = [1, selected_degree, 12]
fig, axes = plt.subplots(1, 3, figsize=(13, 4), sharey=True)

for ax, degree in zip(axes, plot_degrees):
    model = Pipeline([
        ('poly', PolynomialFeatures(degree=degree)),
        ('linear', LinearRegression()),
    ])
    model.fit(X, y)
    y_grid = model.predict(x_grid)

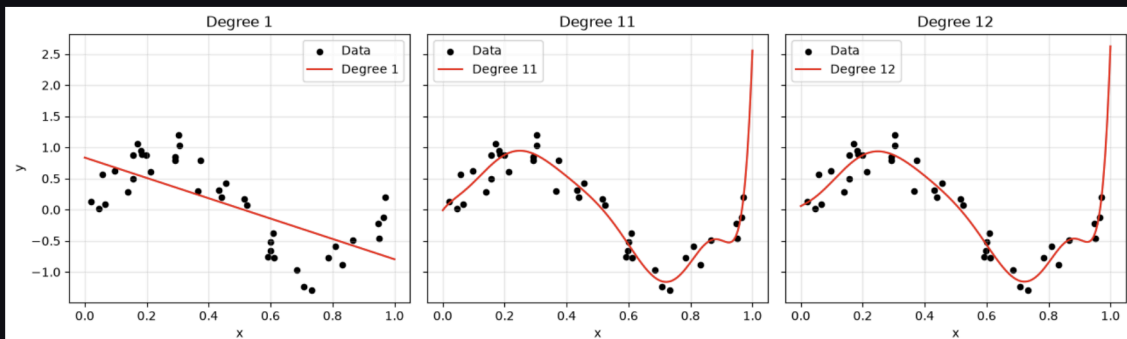
    ax.scatter(X, y, color='black', label='Data', s=20)
    ax.plot(x_grid, y_grid, color='red', label=f'Degree {degree}')
    ax.set_title(f'Degree {degree}')
    ax.set_xlabel('x')
    ax.grid(alpha=0.3)
    ax.legend()

axes[0].set_ylabel('y')
plt.tight_layout()
plt.show()

```

[16]

Python



Part C short answer: Training MSE monotonically decreases as model complexity increases because higher-degree polynomials can memorize noise, making training MSE alone a poor metric for model selection. Degree 1 fails due to high bias (underfitting), as a straight line cannot capture the underlying non-linear sine curve. Degree 12 fails due to high variance (overfitting), wild fluctuations, and high sensitivity to sample noise. Evaluating the one-standard-error rule shows that Degree 3 yields a CV MSE within 1 SE of the winning Degree 4, offering a simpler candidate model.

PART D: CLOSING THE LOOP

Part D short answer: Placing a zero-mean Gaussian prior on the regression weights introduces L2 (Ridge) regularization, which shrinks large parameters toward zero and prevents severe oscillations in the degree-12 curve. In the bias-variance trade-off, this regularization increases the bias slightly by constraining model flexibility. However, it drastically reduces the variance by stabilizing predictions on unseen data.